
UNIT 2 OBJECT ORIENTED PROGRAMMING

Structure

- 2.0 Introduction
- 2.1 Objectives
- 2.2 Principles and Patterns
- 2.3 Python Objects
- 2.4 Abstract Class
- 2.5 Types of Inheritance
- 2.6 Namespace and Object-Orientation
- 2.7 Python Lists
- 2.8 Copy An Object in Python Using '='
- 2.9 Summary
- 2.10 Further Readings

2.0 INTRODUCTION

The object oriented programming (OOP) paradigm is concerned with real world objects and its roots lie in modeling and later writing code around real world objects. These objects are logically linked together to form an internal hierarchy. In programming realization, these real world objects are represented by classes and the hierarchy defines the relationship between these classes. A programmer chooses the data representation that best fits the requirements. Thus a program is structured around the data rather than operations.

In the very simplest terms, objects can be defined as reusable software components that model items in the real world. These are meaningful software units such as student objects modeling the general characteristics of a student, date objects, time objects, invoice objects, etc. These are reusable and more understandable, better organized and modular and easier to maintain than procedural programming.

The main idea behind Procedural Programming is to decompose a task into further subtasks. This process of decomposition is repeated until the subtasks can be appropriately coded. A solution to a problem is thus a conglomerate of functions and subprograms, communicating through arguments and variables that may have a global or local scope. Thus a program is designed and structured based on control flow rather than data flow giving more weightage to program than data.

2.1 OBJECTIVES

The purpose of this unit is to introduce Object Oriented Programming as Python is an Object Oriented Programming language.

After studying this unit, you should be able to understand :

- OOPs concepts
- Python as OOP language

2.2 PRINCIPLES AND PATTERNS

Principles and patterns are useful in defining fundamental design concepts in object-oriented programming (OOP). These act as basis for the design and implementation of robust and maintainable programs. Lets first understand the necessary principles of designing classes in Python.

- **Designing Classes & Class Hierarchies**

Defining and defining any class or class hierarchies requires one to follow certain principles in order to produce a program code that is not only efficient but also minimizes overlap of functionalities among classes and redundancies. Certain important principles that one should follow are listed here.

- **Removing Duplicity of Data and Function**

Functions defined in one class should not be duplicated in other classes. If one has defined a function named “area” for calculating area of a room, then one need not define the same function again as it can be available through inheritance discussed in the sections below.

- **Objects as Real World Entities**

Every real world entity should be represented by a distinct class in the program. This will ensure that the design is efficient with well connected components. Ensure that a class does not pack too much functionality. This makes understandability and maintainability difficult since it demands more cognition. Frequently needed computations qualify for being a function in a class.

- **Access Mechanisms – Public, Protected & Private**

Class Variables are the variables that are shared across all instances of a class. It is defined at the class level, outside any methods. Instance Variables are unique to each instance (object) of a class that are defined within the `__init__` method or other instance methods.

- **Public**

Public members namely attributes / variables and methods/functions are accessible from anywhere, both within and outside the class. By default, all attributes / variables and methods/functions in a Python class are public.

- **Protected**

Protected members are intended for use within the class and its subclasses. They are denoted by a single underscore prefix `_` like “`_protected_method()`”.

- **Private**

Private members are intended for use only within the class. They are denoted by a double underscore prefix `__` like “`__private_method()`”.

Classes are created by keyword `class`. Attributes are the variables that belong to a class. These can be accessed using the dot (`.`) operator.

2.3 PYTHON OBJECTS

An Object is an instance of a Class in Python as is the case with other object oriented programming languages. It represents a specific implementation of the class and holds its own data. An object consists of:

Identity provides a unique name to an object and enables one object to interact with other objects.

State represented by the attributes and exposes the properties of an object.

Behavior represented by the methods of an object and provides the response of an object to other objects.

Having deliberated upon the principles of designing classes in Python, let us explore about the core principles of OOP in Python.

Encapsulation refers to bringing together data / attributes and methods / functions defined within a class. This principle restricts access to certain components to control interactions by the way these are defined within a class. Hence three types of encapsulations practiced in Python are as under:

- **Public Members:** These are accessible from anywhere in the program.
- **Protected Members:** These are accessible only within the class and its subclasses.
- **Private Members:** These are only accessible within the class.

Bundling data (attributes) and methods that operate on that data within a class, controlling access and preventing unintended modification.

Inheritance is fundamental to object oriented programming wherein a new class, known as a child or derived class, acquires attributes and methods from an existing class. This existing class is known as the base class or parent class. This very property of OOP promotes code reuse and facilitates creation of hierarchical relationships between parent and derived classes. The derived class can inherit and use the properties and behaviors of the base class and also extend or modify them. But, this depends on the type of encapsulation.

The newly created class is known as the subclass also referred to as child or derived class. The existing class from which the child class inherits is known as the superclass also referred to as parent or base class. Python supports both single and multiple inheritances. The same is illustrated by a pseudo-code here.

```
# define a superclass
class super_class:
    # attributes and method definition

# Sub_Class Inheriting the Super_Class
class sub_class(super_class):
    # attributes and method of super_class are inherited
    # attributes and method of sub_class also available
```

Polymorphism is a property that allows methods to carry same name as in super class but have different functionality based on the object's context. This is achieved

through method overriding or overloading. Polymorphism facilitates dynamic method usage, where the specific method to be executed is determined at runtime based on the object's actual type. This actually translates into a fact that one can call a method on an object without knowing its exact type in advance. Polymorphism facilitates that the system will automatically choose the appropriate implementation based on the object's class. The two basic types of Polymorphism are discussed in this section.

Compile-time polymorphism also referred to as static polymorphism or method overloading, is a behavior where the specific method to be executed is determined at compile time. Although Python does not support traditional method overloading in the same way that languages like C++ or Java, but similar behaviour also exists through techniques like default arguments or variable-length argument lists.

In languages that provide built-in compile-time polymorphism, multiple methods with the same name can be defined by the programmer but varying parameter lists that could be either the number of parameters or the type of parameters. It is the compiler that selects the appropriate method to call based on the arguments provided in the method / function call.

Python, being a dynamically typed language, resolves method calls at runtime also termed as method overloading. If we have multiple methods with same name in a class, the last definition of a function / method shall override any of its previous definitions. The same is illustrated by a python program below.

```
# Python program to illustrate Compile-Time Polymorphism using method
overloading

# First method named sum()
# It accepts two argument and prints the sum of two numbers
# product

def sum(a, b):
    total = a * b
    print(total)

# Second method named sum()
# It accepts three argument and prints the sum of three numbers

def sum(a, b, c):
    total = a * b*c
    print(total)

sum(10, 20)
sum(10, 20, 30)          #second definition of sum() with three parameters
```

➔ Output

Hangup (SIGHUP)

Traceback (most recent call last):

File "/home/guest/sandbox/Solution.py", line 16, in <module>

sum(10, 20)

~~~~~

TypeError: sum() missing 1 required positional argument: 'c'

The error message appears because the last definition of the method `sum()` expects three arguments. When it encounters “`sum(10, 20)`”, the compiler finds that the third argument is missing and hence the error message.

**Run-Time Polymorphism** is a type of polymorphism is determined during the execution of the program. It occurs when a subclass contains a implementation for any / all method(s) already defined in its parent class, commonly known as method overriding. Some commonly practiced polymorphism in Python are the following:

- Polymorphism in Built-in Functions
- Polymorphism in Functions
- Polymorphism in Operators
- Polymorphism in Object-Oriented Programming (OOP)

#### # Python program to illustrate Polymorphism in Functions

```
print(len("Good Morning"))      # String length
print(len([10, 20, 30, 40]))    # List length
```

➔ Output

```
12
4
```

#### # Python program to illustrate Polymorphism in Functions

```
def add(a, b):
    return a + b

print(add(10, 20))              # Integer addition
print(add("Good ", "Morning"))  # String concatenation
print(add([10, 20], [30, 40]))  # List concatenation
```

➔ Output

```
30
Good Morning
[10, 20, 30, 40]
```

Operators like `+` behave polymorphically thereby carrying out addition, concatenation or merging of lists based on the data type of the parameters.

#### # Python program to illustrate Polymorphism in Operators

```
print(10 + 20)                  # Integer addition
print("Good " + "Morning")      # String concatenation
print([10, 20] + [30, 40])      # List concatenation
```

➔ Output

```
30
Good Morning
[10, 20, 30, 40]
```

As with other object oriented programming languages, polymorphism in python allows methods in different classes to share the same name but perform different tasks. This is achieved through inheritance and interface design. Polymorphism complements other OOP principles like inheritance (sharing behavior) and

encapsulation (hiding complexity) to create robust and modular applications. It can be seen from the program below that we have not created any common superclass for inheritance or linked the classes together in any form. Still, using polymorphism, we have placed these two different objects namely Faculty and Student into a tuple and iterated through it using a common animal variable as illustrated in the statement “for person in (Faculty1, Student1, Student2)”.

### # Python program to illustrate Polymorphism in Class Methods

class Faculty:

```
def __init__(self, name, age):
    self.name = name
    self.age = age

def info(self):
    print(f"My name is {self.name}. I am {self.age} years old.")
```

class Student:

```
def __init__(self, name, age):
    self.name = name
    self.age = age

def info(self):
    print(f"My name is {self.name}. I am {self.age} years old.")
```

Faculty1 = Faculty("DSK", 55)

Student1 = Student("Karan", 26)

Student2 = Student("Nikita", 30)

```
for person in (Faculty1, Student1, Student2):
    person.info()
```

➔ Output

My name is DSK. I am 55 years old.

My name is Karan. I am 26 years old.

My name is Nikita. I am 30 years old.

**Abstraction** in Python is a way to manage complexity of the program by hiding unnecessary details and showing only essential information. Essentially it means that abstraction is used to hide irrelevant details from the user and show the details that are relevant to the users. It is one of the core principles of Object-Oriented Programming (OOP) paradigm. Abstraction helps in creating a simplified view of objects, making code more manageable and understandable. This property simplifies interaction with objects.

Python also supports abstract and concrete classes.

---

## 2.4 ABSTRACT CLASS

---

An abstract class are not instantiated directly as these serve as a blueprint or template for other classes. It can contain abstract methods that exist without

implementation and concrete methods that are implemented. These abstract classes are created using the abc module and the ABC class. Any subclass that inherits from an abstract class must implement its abstract methods.

A concrete class can be instantiated directly and it inherits methods from an abstract class that are implemented here. A concrete class provides specific implementations for all necessary methods.

```
from abc import ABC, abstractmethod # Import required modules
class abstraction(ABC):              # Creates Abstract base class
    @abstractmethod                  # Declares abstract method
    def abs_method(self):
        print ("This is abstract method")
        return
    def con_method(self):
        print ("This is concrete method")

class concreteclass(abstraction):    # Creates a child class
    def abs_method(self):
        super().abs_method()
        return

obj = concreteclass()                # Create an instance of concreteclass
obj.abs_method()
obj.con_method()
```

→ Output

This is abstract method  
This is concrete method

---

## 2.5 TYPES OF INHERITANCE

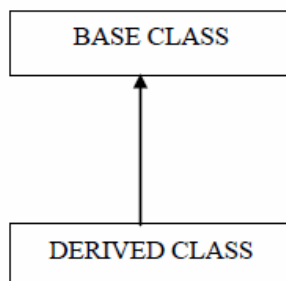
---

Inheritance can be classified on factors based on the number of base class from which properties have been inherited in the inheritance hierarchy or it could be based on combination of these properties. The major types are:

1. Single Inheritance
2. Multiple Inheritance
3. Multilevel Inheritance
4. Hierarchical Inheritance
5. Hybrid Inheritance

### 2.5.1 Single Inheritance

When a class is derived from one base class that is when a class inherits properties and methods from only one parent class, then it is termed as single inheritance. This is the simplest form of inheritance and the inheritance diagram is:



```

# Creating a Parent / Base class
class Parent:
    def ParentMethod(self):
        print ("Johny Calling Method from Parent Class")

# Creating a Child class
class Child(Parent):
    def childMethod(self):
        print ("Johny Calling Method from Child Class")

# Creating Instance of Child
Johny = Child()

# calling method of
Johny.ParentMethod()
print()
# calling method of child class
Johny.childMethod()

```

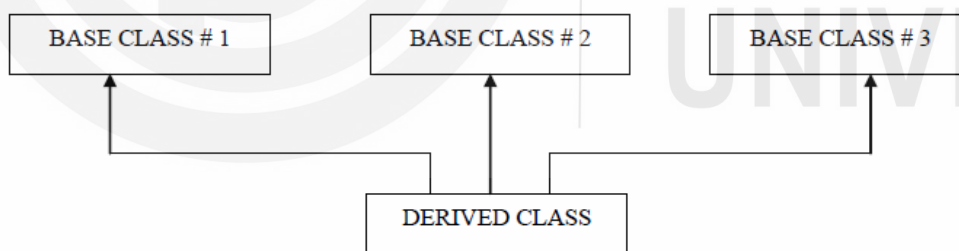
### ➔ Output

Johny Calling Method from Parent Class

Johny Calling Method from Child Class

## 2.5.2 Multiple Inheritance

When a class is derived from two or more base classes, then a class is termed as multiple inheritance. This kind of inheritance allows one to combine features from several existing classes, thus promoting class reuse. This type of inheritance is suitable in larger programs, where complex hierarchies are needed.



```

class Faculty:
    def Emp_code(self):
        return "Employee Code of Faculty"

```

```

class Lab_Attd:
    def Lab_Taken(self):
        return "OOPs Lab"

```

```

class Student(Faculty, Lab_Attd):
    pass

```



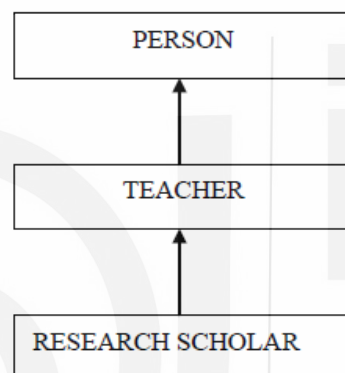
```
# Example usage
student = Student()
print(student.Emp_code())
print(student.Lab_Taken())
```

➔ Output

```
Employee Code of FAculty
OOPs Lab
```

### 2.5.3 Multilevel Inheritance

When a derived class is obtained from a base class which itself is derived from some other super class, it is termed as multilevel inheritance. Code reuse is maximized in this inheritance by using existing functionalities.



```
# Python program to illustrate Multiple Inheritance
```

```
# parent class
```

```
class Faculty:
```

```
    def Emp_code(self):
```

```
        print ("Employee Code of FAculty")
```

```
# Child class 'Lab_Attd' Inheriting methods of parent class 'Faculty'
```

```
class Lab_Attd(Faculty):
```

```
    def Lab_Taken(self):
```

```
        print ("OOPs Lab")
```

```
# Another child class 'Intern' Inheriting methods of parent class 'Lab_Attd'
```

```
class Intern(Lab_Attd):
```

```
    def Quiz(self):
```

```
        print ("Checks Quiz & Assignment")
```

```
# Creating instance of Intern
```

```
person = Intern()
```

```
# method calls
person.Emp_code()
print()
person.Lab_Taken()
print()
person.Quiz()
```

➔ Output

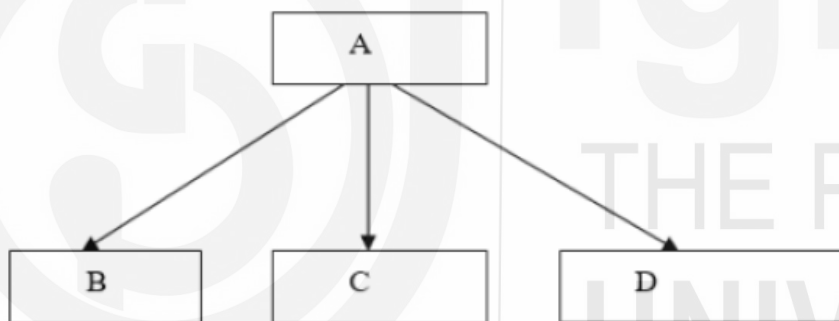
Employee Code of Faculty

OOPs Lab

Checks Quiz & Assignment

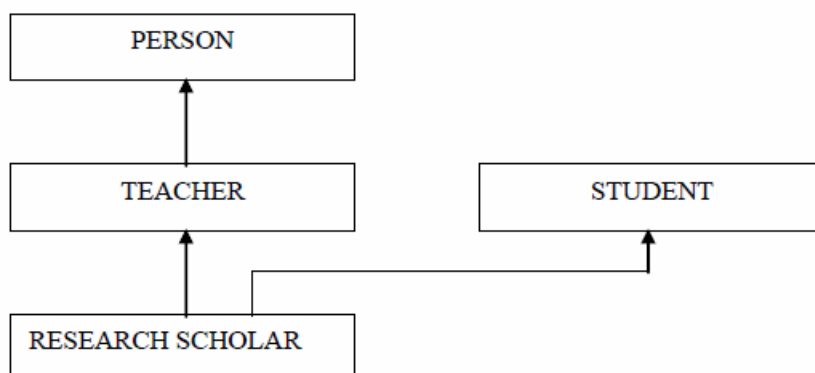
#### 2.5.4 Hierarchical Inheritance

Hierarchical inheritance is a design where multiple child classes inherit from a single parent class. Apparently it shall appear that it promotes strong code reuse, it also leads to tight coupling between the parent and child classes. Hence, any change done in the parent class can have a ripple effect on all child classes that inherit functionalities from these.



#### 2.5.5 Hybrid Inheritance

When a class is derived with a mix of two or more types of inheritance, it is derived as hybrid inheritance.



Here the class Research scholar is derived using multilevel inheritance and single inheritance. Hence this inheritance represents hybrid inheritance. This enables to merge functionalities from multiple parent classes, thus inheriting features from various parents. As an example, suppose we have well defined classes for ICE Car and

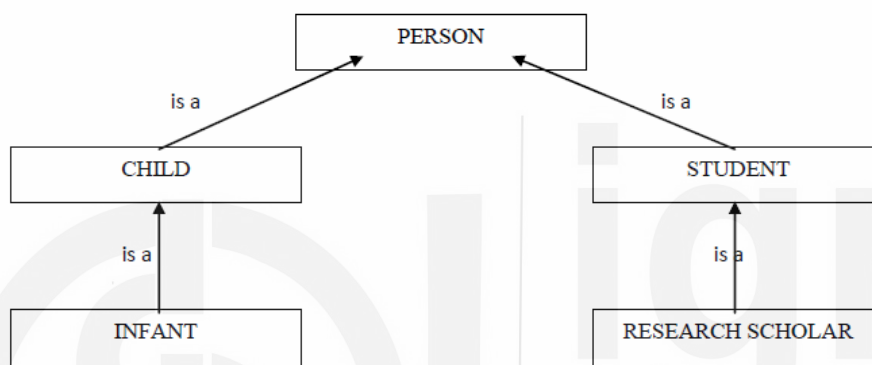
Electric car, one can inherit functionalities from both the classes to create a Hybrid Car class.

### 2.5.6 Benefits, Costs and Implications of Inheritance

One can use inheritance to create a specialized version from a general purpose base class. Suppose one has a general purpose base class “CYLINDER”. This base class could include members such as radius and diameter of cylinder, volume of cylinder. Once a class is derived from this base class, one can include new member variable such as weight of the cylinder or height of cylinder. Thus it is helpful in SPECIALIZATION.

On the other hand, one could inheritance to extend the functionality if a base class, such that the derived class might include some new member functions or new data types. This helps in extending the capability of a class.

Inheritance is also used to represent ‘Is a Relationship’ among various types of entities.



Among the many reasons, one of the important reason for deriving a sub-class from a base class is to model a new type of object in terms of one or more existing types or to extend the functionality of an existing class i.e. specialization. This specialization is done by adding new member data or member functions.

---

## 2.6 NAMESPACE AND OBJECT-ORIENTATION

---

A namespace is a collection of names where we have a mapping of every name we have defined to corresponding objects in the code. An object can be a variable or a method. Name represents a unique identifier and Space refers to the scope of the name. Python itself maintains a namespace in the form of a Python dictionary. Namespaces are crucial in OOP for organizing and accessing class-level and instance-level variables and methods. As discussed in Chapter 1, each class and object has its own namespace, preventing naming conflicts and ensuring proper access to attributes and methods of the objects. The two basic types of these namespaces are:

- Class Namespace that contains attributes and methods defined within a class that is shared by all instances of the class.
- Instance Namespace contains attributes specific to each instance of a class.

---

## 2.7 PYTHON LISTS

---

A list in Python is a built-in dynamic sized array that automatically grows and shrinks. The list can store various types of data items including another list within a list. This feature is made possible because a list only stores references at contiguous locations and actual items may be stored at different locations. A list can also

contain duplicate items. Accessing any data items in List can be done directly using their subscript or index that starts from 0.

### Creating a List in Python

Lists in Python can be created either by using Square Bracket or Using list() Constructor

#### # Creating Python List using Square Bracket []

```
# List of integers
a = [10, 20, 30, 40, 50]

# List of strings
b = ['C++', 'Java', 'Python']

# Mixed data types
c = [10, 'C++', 100, 'False']

print(a)
print(b)
print(c)
print(c[3])
```

➔ Output

```
[10, 20, 30, 40, 50]
['C++', 'Java', 'Python']
[10, 'C++', 100, 'False']
False
```

#### # Creating a Python List Using list() Constructor

```
a = list((10, 'C++', 100, 'False'))

print(a)
```

➔ Output

```
[10, 'C++', 100, 'False']
```

#### # Creating Empty List & Inserting Elements

```
# Initialize an empty list
emptylist = []

# Inserting 5 at index 0
emptylist.insert(0, 5)
print("After Insert(0, 5) operation:", emptylist)
```

➔ Output

```
After Insert(0, 5) operation: [5]
```

#### # Accessing Elements of the List by their Indexes

```
print(a[0]) # 5
print(a[1]) # "List"
print(a[2]) # 15
```

```
print(a[3]) # 25
print(a[4]) # True
```

# Checking for Data Types of list elements

```
print(type(a[1])) # str
print(type(a[4])) # bool
```

➔ Output

```
[5, 'List', 15, 25, True]
5
List
15
25
True
<class 'str'>          # Prints the data type of element at index 1
<class 'bool'>        # Prints the data type of element at index 4
```

# Python Program to Access last Element of List

```
# List of integers
a = [10, 20, 30, 40, 50]

print(a[-1])
```

➔ Output

```
50
```

**Adding Elements into a List using Append & Extend**

```
# Python List of integer Numbers
a = [10, 20, 30, 40, 50]

print(a[-1])
a.append(60)
print("List after append(10):", a)
a.extend([70, 80, 90])
print("List after extend(10):", a)
```

➔ Output

```
50
List after append(10): [10, 20, 30, 40, 50, 60]
List after extend(10): [10, 20, 30, 40, 50, 60, 70, 80, 90]
```

---

## 2.8 COPY AN OBJECT IN PYTHON USING ‘=’

---

Python makes use of ‘=’ operator to create a copy of an object. One needs to understand that it does not create a new object, but it only creates a new variable that shares the reference of the original object. The same is illustrated by a python program below where we have created a list named “original\_list” and we pass its object reference to “copied\_list” using the ‘=’ operator.

### 2.8.1 Copy an Object in Python using Copy() Method

Python lists are useful data structures used to store a collection of items. The `copy()` list method is important when working with data since it helps preserve the original list while performing operations on the duplicated one. This prevents from any untoward data loss.

The `copy()` method is a built-in method in Python that is used to create a shallow copy of the list. The `copy()` method has been made available only from Python 3.3 version onwards. This method copies only the top-level list structure. The syntax of the `copy()` method is shown below.

### 2.8.2 Copy a List in Python

```
# Define the original list of Elements
original_list = [10, 20, 30, 40]
```

```
# Creating a Shallow copy of the Original List
copied_list = original_list.copy()
```

```
# Modifying the first element of the Copied List
copied_list[0] = 99
```

```
print(original_list)
print(copied_list)
```

➔ Output

```
[10, 20, 30, 40]
[99, 20, 30, 40]
```

#### Shallow and Deep Copying

A shallow copy in Python doesn't create a copy of objects, instead it just copies the reference of the objects. A deep copy creates a new object and recursively adds the copies of objects present in the original elements. Hence a new object with all the list elements is created in the deep copying operation.

#### The Copy Module of Python

One has to make use the `copy` module of Python for shallow and deep copy operations. Suppose, you need to copy the compound list say `elementlist`. Then one makes use of the following:

```
import copy
copy.copy(elementlist)      # Shallow Copy
copy.deepcopy(elementlist)  # Deep Copy
```

#### Creating a Shallow Copy of the List

```
import copy
```

```
original_list = [[10, 20, 30], [40, 50, 60]]
copied_list = copy.copy(original_list)
```

```
print("Original list:", original_list)
print("Copy of Original list:", copied_list)
```

➔ Output

```
Original list: [[10, 20, 30], [40, 50, 60]]
Copy of Original list: [[10, 20, 30], [40, 50, 60]]
```

Now if we change the third element of the copied\_list to 100, we obtain the following output.

```
copied_list[0][2] = 100

print("Original list:", original_list)
print("Copy of Original list:", copied_list)
```

#### ➔ Output

```
Original list: [[10, 20, 100], [40, 50, 60]]
Copy of Original list: [[10, 20, 100], [40, 50, 60]]
```

It is evident from the above piece of code that by changing a nested / child element of the copied\_list, the same affect gets reflected in the original\_list. This is because the object resident in the dynamic memory is referenced by the pointers by both the original\_list and the copied\_list. Hence any change made in the copied\_list actually changes the value of the object resident in the dynamic memory or heap. In order to avoid this, we need to make use of deep copy.

### 2.8.3 Deep Copy

A deep copy carries out a recursive copying process of the original collection object creates a copy of the object, and it does so by populating copies of the child objects found in the original. That means that the new collection object (copied\_list) and its elements are independent of the original (original\_list).

```
import copy

original_list = [[10, 20, 30], [40, 50, 60]]
copied_list = copy.deepcopy(original_list)

print("Original list:", original_list)
print("Deep Copy of Original list:", copied_list)

print()

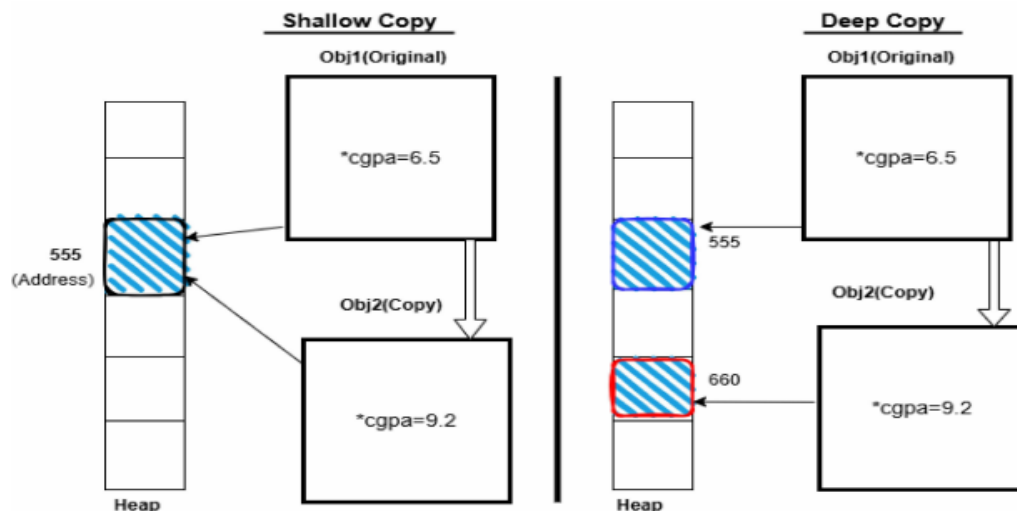
copied_list[0][2] = 100

print("Original list:", original_list)
print("Deep Copy of Original list:", copied_list)
```

#### ➔ Output

```
Original list: [[10, 20, 30], [40, 50, 60]]
Deep Copy of Original list: [[10, 20, 30], [40, 50, 60]]

Original list: [[10, 20, 30], [40, 50, 60]]
Deep Copy of Original list: [[10, 20, 100], [40, 50, 60]]
```



The concepts of shallow and deep copying have been illustrated in the figure. Now it should be amply clear that the Shallow copies are efficient and faster because it uses less heap memory but can lead to unintended changes. Deep copies use more memory and processing time but ensure the copied list is fully independent and any change made in the copy shall not affect the original.

However one needs to exercise certain cautions. Recursive objects also referred to as compound objects that, directly or indirectly, contain a reference to themselves may cause a recursive loop. Also, since deep copy copies everything it may copy too much, such as data which is meant to be shared between the copies of an object.

## 2.9 SUMMARY

Object-oriented programming paradigm provides complete insulation from internal architecture and operations. It provides a very high level of abstraction in problem solving. Programs are designed with a focus on data around which the code is built and this complete entity is known as object. This is a more realistic way of representing the world in general. Python as Object-Oriented Programming Language is to write structured programs by creating "objects," that encapsulate data or attributes and functions or methods that operate on that data. This improves the program structure and enhances code reusability. Major types of Inheritance supported by Python are Single inheritance, Multiple Inheritance, Multilevel Inheritance, Hierarchical Inheritance and Hybrid Inheritance.

Polymorphism allows methods carry the same method name as in super class but different functionality based on the object's context. This is achieved through method overriding or overloading. Polymorphism facilitates dynamic method usage, where the specific method to be executed is determined at runtime based on the object's actual type. Abstraction helps to manage complexity of the program by hiding unnecessary details and showing only essential information to the programmer.

An abstract class are not instantiated directly as these serve as a blueprint or template for other classes. Abstract methods exist without its implementation and only the concrete methods that are implemented. A concrete class can be instantiated directly and it inherits methods from an abstract class that are implemented here. Among the many reasons, one of the important reasons for deriving a sub-class from a base class is to model a new type of object in terms of one or more existing types or to extend the functionality of an existing class i.e. specialization.

Namespaces are crucial in OOP for organizing and accessing class-level and instance-



level variables and methods. As discussed in Chapter 1, each class and object has its own namespace, preventing naming conflicts and ensuring proper access to attributes and methods of the objects. The two basic types of these namespaces are Class Namespace that contains attributes and methods defined within a class that is shared by all instances of the class and Instance Namespace contains attributes specific to each instance of a class.

A list in Python is a built-in dynamic sized array that automatically grows and shrinks. The list can store various types of data items including another list within a list. A list can also contain duplicate items. Accessing any data items in List can be done directly using their subscript or index that starts from 0.

Python makes use of '=' operator to create a copy of an object. One needs to understand that it does not create a new object, but it only creates a new variable that shares the reference of the original object. A shallow copy in Python doesn't create a copy of objects, instead it just copies the reference of the objects. A deep copy creates a new object and recursively adds the copies of objects present in the original elements. Deep copies use more memory and processing time but ensure the copied list is fully independent and any change made in the copy shall not affect the original.

### Check your Progress

- Q1. How can one concatenate two lists in Python?
- Q2. What is pass in Python coding?
- Q3. What are Modules and Packages in Python?
- Q4. What is the difference between a shallow copy and a deep copy?
- Q5. Does Python support multiple Inheritance?
- Q6. What is the significance of Polymorphism in Python?
- Q7. What is \_\_init\_\_() in Python and how does self play a role in it?
- Q8. What is init method in python?
- Q9. Is it possible to call parent class without creating its instance?
- Q10. What is the significance of inheritance in python?
- Q11. Why does one use else in try/except construct in Python?

---

## 2.10 FURTHER READINGS

---

1. Python Essentials You Always Wanted to Know: Beginner's Guide to Python Programming, Data Structures, Data Analytics with Hands-On Coding Exercises by Shawn Peters and Vibrant Publishers
2. Python Programming : Beginners Guide Paperback by Sundarajan M (Author), Mani Deepak Choudhry (Author), Jeevanandham S (Author), Akshya Jothi (Author)
3. The easy basics of Python: Getting started with object-oriented programming using Python by [Nicolas Snake](#) (ebook)

## Reference Websites

<https://pythoninstitute.org/python-essentials-1>

<https://pythoninstitute.org/python-essentials-2>



ignou  
THE PEOPLE'S  
UNIVERSITY